САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание №5

Выполнил:

Федоров Даниил

Группа
K3440

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

- подключить и настроить rabbitMQ/kafka;
- реализовать межсервисное взаимодействие посредством rabbitMQ/kafka.

Ход работы

Подключаем RabbitMQ

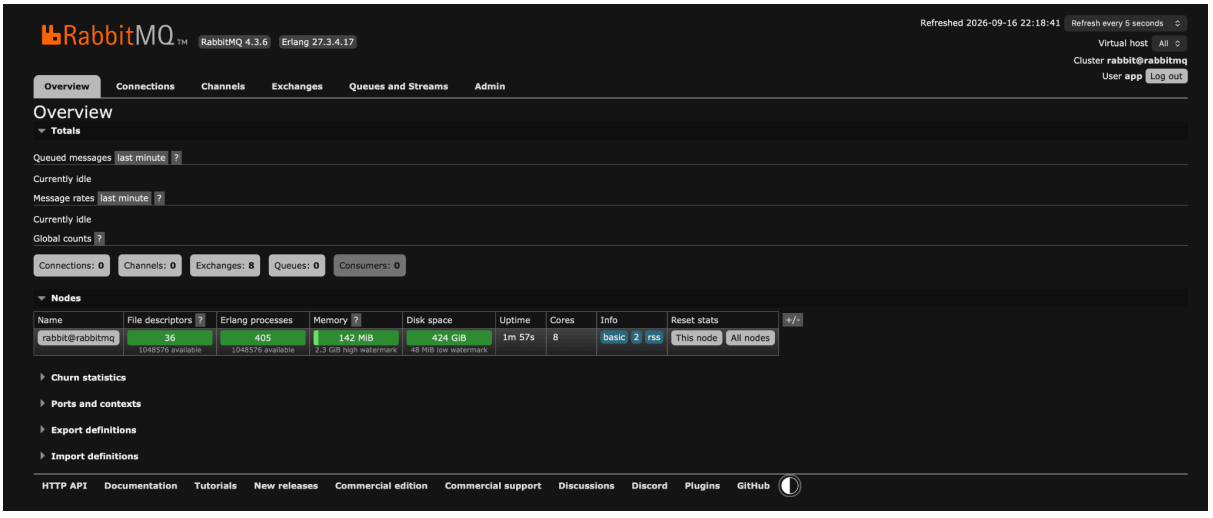
Добавляем RabbitMQ в docker-compose.yml

```yaml
docker-compose.yml
▷Run All Services
services:
  ▷Run Service
  db:
    image: postgres:16
    restart: always
    ports:
      - 15432:5432
    volumes:
      - ./dbs/postgres-data:/var/lib/postgresql/data
    env_file:
      - .env

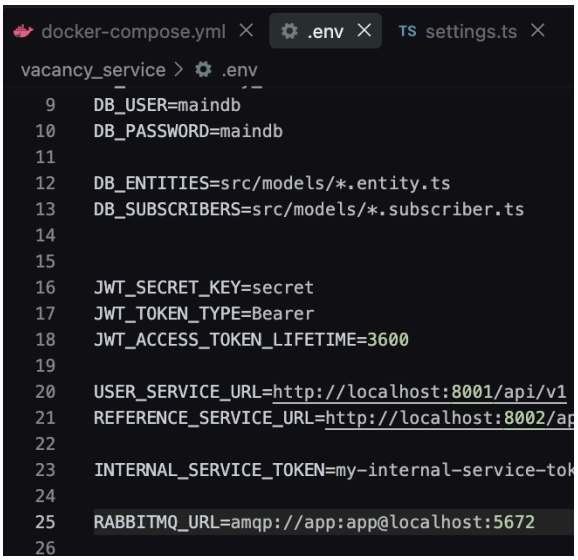
  ▷Run Service
  rabbitmq:
    image: rabbitmq:4-management
    restart: always
    hostname: rabbitmq
    ports:
      - "5672:5672"
      - "15672:15672"
    environment:
      RABBITMQ_DEFAULT_USER: app
      RABBITMQ_DEFAULT_PASS: app
    volumes:
      - rabbitmq-data:/var/lib/rabbitmq

volumes:
  rabbitmq-data:
```

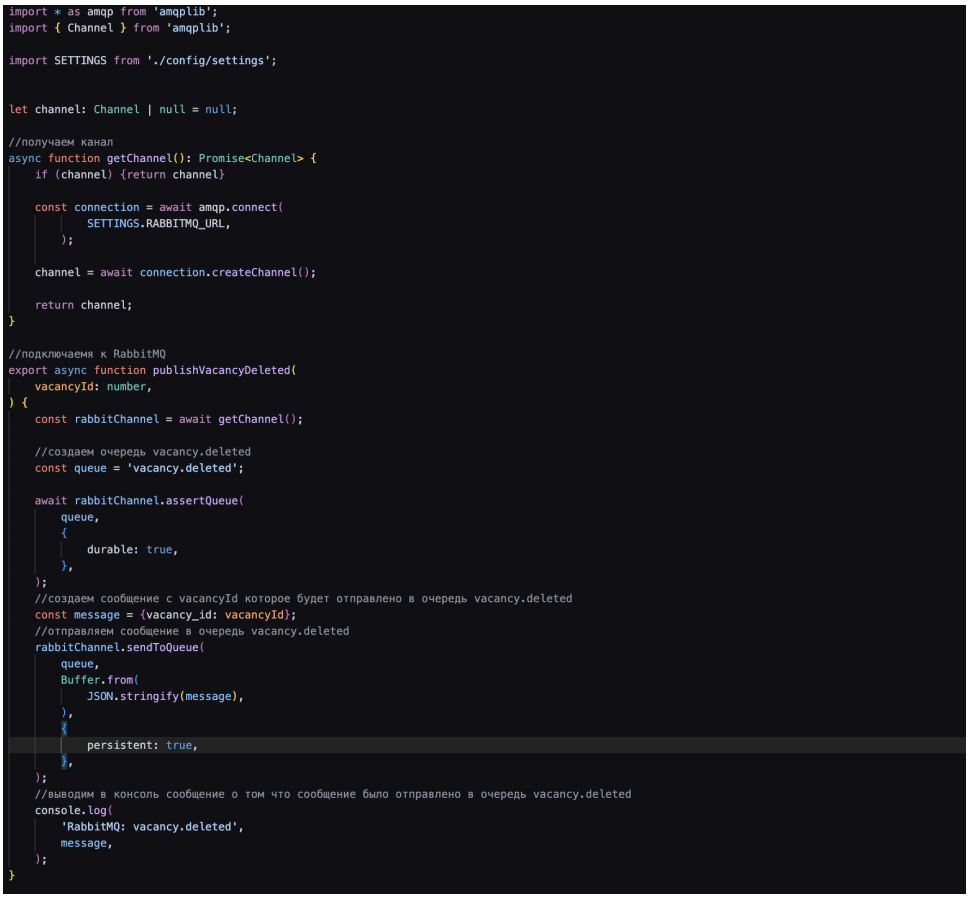
Далее запускаем и проверяем что rabbit mq management работает



Для начала установим библиотеку в vacancy service и добавим RABBITMQ_URL в settings и .env в этом микросервисе



Далее создадим файл RabbitMQ в src микросервиса. Этот файл работает по простой логике: он подключается к RabbitMQ, создает очередь vacancy.deleted, и отправляет в нее сообщение с vacancyId.



Далее нам следует подключить его к удалению вакансии. У нас получается что при удалении вакансии мы отправляем сообщение в RabbitMQ о том что вакансия удалена, и это сообщение будет обработано другими сервисами которые подписаны на очередь vacancy.deleted

```
    @Delete('/employers/:employerId/vacancies/:vacancyId')
    @OnUndefined(204)
    @UseBefore(authMiddleware, allowRoles(UserRole.EMPLOYER))
    async deleteVacancy(
        @Param('employerId')
        employerId: number,

        @Param('vacancyId')
        vacancyId: number,

        @Req()
        request: RequestWithUser,
    ) {
        this.ensureEmployerOwner(request, Number(employerId));

        const vacancy =
            await this.vacancyRepository.findOne({
                where: {
                    vacancyId:
                        Number(vacancyId),

                    employerId:
                        Number(employerId),
                },
            });

        if (!vacancy) {throw new NotFoundError('Вакансия не найдена')}

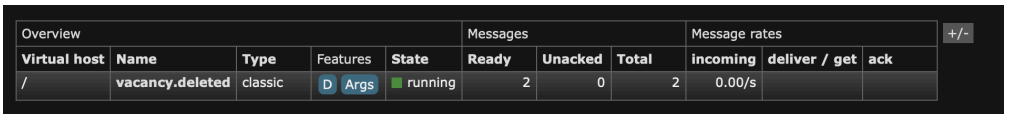
        await this.vacancyRepository.remove(vacancy);

        await publishVacancyDeleted(vacancy.vacancyId);

        return undefined;
    }
}

export default VacanciesController;
```

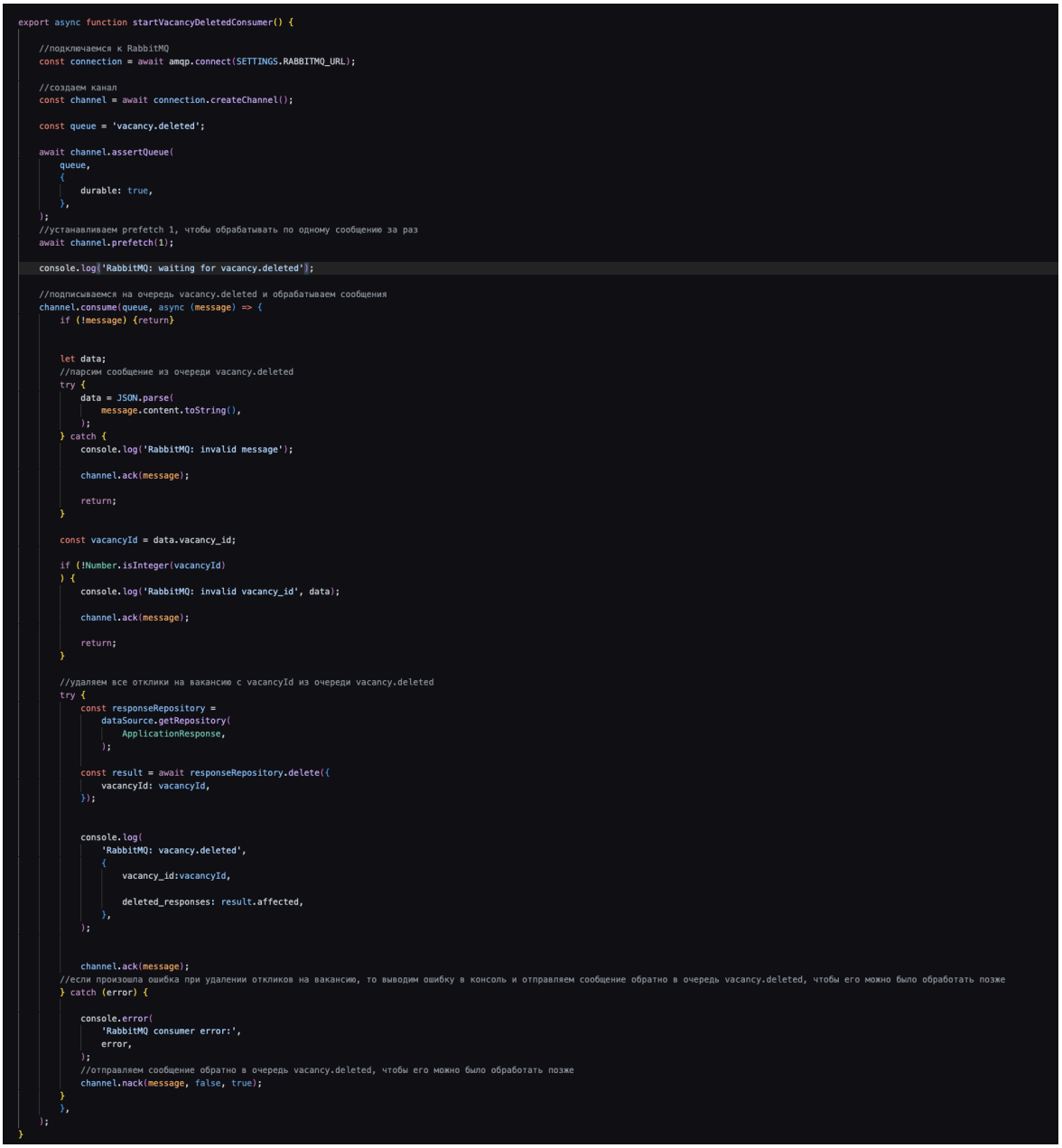
Далее проверка, я создал и удалил две вакансии и они поместились в очередь



Overview					Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	vacancy.deleted	classic	D Args	running	2	0	2	0.00/s			

Сейчас мы удалили из одного микросервиса вакансии, но отклики связанные с ними не удалились. Следующим шагом создадим файл который будет слушать очередь и удалять отклики в response_service.

Создаем consumer. Работает по простой логике: когда приходит сообщение о том что вакансия удалена, мы удаляем все отклики на эту вакансию из базы данных. Если при удалении откликов произошла ошибка, то мы отправляем сообщение обратно в очередь vacancy.deleted, чтобы его можно было обработать позже.



```
export async function startVacancyDeletedConsumer() {

    //подключаемся к RabbitMQ
    const connection = await amqp.connect(SETTINGS.RABBITMQ_URL);

    //создаем канал
    const channel = await connection.createChannel();

    const queue = 'vacancy.deleted';

    await channel.assertQueue(
        queue,
        {
            durable: true,
        },
    );
    //устанавливаем prefetch 1, чтобы обрабатывать по одному сообщению за раз
    await channel.prefetch(1);

    console.log('RabbitMQ: waiting for vacancy.deleted');

    //подписываемся на очередь vacancy.deleted и обрабатываем сообщения
    channel.consume(queue, async (message) => {
        if (!message) {return}


        let data;
        //парсим сообщение из очереди vacancy.deleted
        try {
            data = JSON.parse(
                message.content.toString(),
            );
        } catch {
            console.log('RabbitMQ: invalid message');

            channel.ack(message);

            return;
        }

        const vacancyId = data.vacancy_id;

        if (!Number.isInteger(vacancyId)
        ) {
            console.log('RabbitMQ: invalid vacancy_id', data);

            channel.ack(message);

            return;
        }

        //удаляем все отклики на вакансию с vacancyId из очереди vacancy.deleted
        try {
            const responseRepository =
                dataSource.getRepository(
                    ApplicationResponse,
                );

            const result = await responseRepository.delete({
                vacancyId: vacancyId,
            });

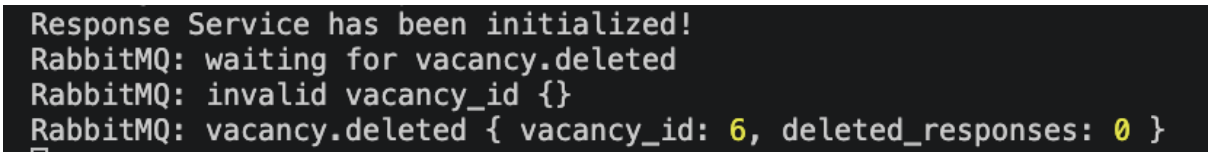

            console.log(
                'RabbitMQ: vacancy.deleted',
                {
                    vacancy_id:vacancyId,

                    deleted_responses: result.affected,
                },
            );


            channel.ack(message);
        //если произошла ошибка при удалении откликов на вакансию, то выводим ошибку в консоль и отправляем сообщение обратно в очередь vacancy.deleted, чтобы его можно было обработать позже
        } catch (error) {

            console.error(
                'RabbitMQ consumer error:',
                error,
            );
            //отправляем сообщение обратно в очередь vacancy.deleted, чтобы его можно было обработать позже
            channel.nack(message, false, true);
        }
        },
    );
}
```

Теперь, когда мы запустили снова микросервис ему передались сообщения.



В рамках работы RabbitMQ был применён только для события удаления вакансии (vacancy.deleted). Это связано с тем, что в данном сценарии Vacancy Service не требуется немедленный ответ от Response Service: после удаления вакансии достаточно уведомить другой сервис о произошедшем событии, а удаление связанных откликов может быть выполнено асинхронно.

Остальные межсервисные взаимодействия были оставлены через REST, поскольку для них результат нужен сразу. Например, при создании отклика необходимо немедленно проверить существование вакансии и резюме. Использование RabbitMQ в таких случаях усложнило бы взаимодействие без практической необходимости.

Таким образом, брокер сообщений был использован только там, где асинхронная обработка действительно оправдана.

Вывод

В ходе работы в микросервисную архитектуру был добавлен брокер сообщений RabbitMQ. Был реализован асинхронный обмен сообщениями между Vacancy Service и Response Service при удалении вакансии. Vacancy Service публикует событие vacancy.deleted, а Response Service получает его и удаляет связанные отклики из своей базы данных. В результате была уменьшена связность между сервисами и реализована асинхронная обработка событий без замены синхронных REST-запросов там, где требуется немедленный ответ.